

Day 42 – Tree Practice (Important Questions)

◆ 1. Left View of Binary Tree

👉 Har level ka first node print karo

💻 Code (Java)

```

1. package Day_42;
2.
3. import java.util.*;
4.
5. class Treeleft {
6.     static class Node {
7.         int data;
8.         Node left, right;
9.         Node(int data) {
10.            this.data = data;
11.            left = right = null;
12.        }
13.    }
14.
15.    static void leftView(Node root) {
16.        if (root == null) return;
17.
18.        Queue<Node> q = new LinkedList<>();
19.        q.add(root);
20.
21.        while (!q.isEmpty()) {
22.            int size = q.size();
23.
24.            for (int i = 0; i < size; i++) {
25.                Node curr = q.poll();
26.
27.                if (i == 0)
28.                    System.out.print(curr.data + " ");
29.
30.                if (curr.left != null) q.add(curr.left);
31.                if (curr.right != null) q.add(curr.right);
32.            }
33.        }
34.    }
35.
36.    public static void main(String[] args) {
37.        Node root = new Node(1);
38.        root.left = new Node(2);
39.        root.right = new Node(3);
40.        root.left.left = new Node(4);
41.        root.left.right = new Node(5);
42.
43.        leftView(root);
44.    }
45. }
46.

```

✅ Output:

1 2 4

◆ 2. Right View of Binary Tree

👉 Har level ka last node print karo

```

1. package Day_42;
2.
3. import java.util.*;
4.
5. class Treeright {
6.     static class Node {
7.         int data;
8.         Node left, right;
9.         Node(int data) {
10.            this.data = data;
11.            left = right = null;
12.        }
13.    }
14.
15.    static void rightView(Node root) {
16.        if (root == null) return;
17.
18.        Queue<Node> q = new LinkedList<>();
19.        q.add(root);
20.
21.        while (!q.isEmpty()) {
22.            int size = q.size();
23.
24.            for (int i = 0; i < size; i++) {
25.                Node curr = q.poll();
26.
27.                if (i == size - 1)
28.                    System.out.print(curr.data + " ");
29.
30.                if (curr.left != null) q.add(curr.left);
31.                if (curr.right != null) q.add(curr.right);
32.            }
33.        }
34.    }
35.
36.    public static void main(String[] args) {
37.        Node root = new Node(1);
38.        root.left = new Node(2);
39.        root.right = new Node(3);
40.        root.left.left = new Node(4);
41.        root.left.right = new Node(5);
42.
43.        rightView(root);
44.    }
45. }
46.

```

✅ Output:

2 3 5

◆ 3. Lowest Common Ancestor (LCA)

👉 Do nodes ka common parent

```

1. package Day_42;
2.
3. class TreeLca {
4.
5.     // Node class
6.     static class Node {
7.         int data;
8.         Node left, right;
9.
10.        Node(int data) {
11.            this.data = data;
12.            left = right = null;
13.        }
14.    }
15.
16.    // LCA Function
17.    static Node LCA(Node root, int n1, int n2) {
18.        if (root == null) return null;
19.
20.        if (root.data == n1 || root.data == n2)
21.            return root;
22.
23.        Node left = LCA(root.left, n1, n2);
24.        Node right = LCA(root.right, n1, n2);
25.
26.        if (left != null && right != null)
27.            return root;
28.
29.        return (left != null) ? left : right;
30.    }
31.
32.    public static void main(String[] args) {
33.
34.        /*
35.           1
36.          / \
37.         2   3
38.        / \ / \
39.       4  5 6  7
40.        */
41.
42.        Node root = new Node(1);
43.        root.left = new Node(2);
44.        root.right = new Node(3);
45.
46.        root.left.left = new Node(4);
47.        root.left.right = new Node(5);
48.
49.        root.right.left = new Node(6);
50.        root.right.right = new Node(7);
51.
52.        int n1 = 4;
53.        int n2 = 5;
54.
55.        Node lca = LCA(root, n1, n2);
56.
57.        if (lca != null)
58.            System.out.println("LCA of " + n1 + " and " + n2 + " is: " + lca.data);
59.        else
60.            System.out.println("LCA not found");
61.    }
62. }
63.

```

◆ 4. Zig-Zag Traversal

👉 Level order but alternate direction

```

1. package Day_42;
2.
3. import java.util.*;
4.
5. class ZigZagTree {
6.
7.     // Node class
8.     static class Node {
9.         int data;
10.        Node left, right;
11.
12.        Node(int data) {
13.            this.data = data;
14.            left = right = null;
15.        }
16.    }
17.
18.    // ZigZag Function
19.    static void zigzag(Node root) {
20.        if (root == null) return;
21.
22.        Queue<Node> q = new LinkedList<>();
23.        q.add(root);
24.
25.        boolean leftToRight = true;
26.
27.        while (!q.isEmpty()) {
28.            int size = q.size();
29.            List<Integer> list = new ArrayList<>();
30.
31.            for (int i = 0; i < size; i++) {
32.                Node curr = q.poll();
33.                list.add(curr.data);
34.
35.                if (curr.left != null) q.add(curr.left);
36.                if (curr.right != null) q.add(curr.right);
37.            }
38.
39.            // Reverse alternate levels
40.            if (!leftToRight)
41.                Collections.reverse(list);
42.
43.            System.out.println(list);
44.
45.            leftToRight = !leftToRight;
46.        }
47.    }
48.
49.    public static void main(String[] args) {
50.
51.        /*
52.           1
53.          / \
54.         2   3
55.        / \ / \
56.       4  5 6  7
57.        */
58.
59.        Node root = new Node(1);
60.        root.left = new Node(2);
61.        root.right = new Node(3);
62.
63.        root.left.left = new Node(4);
64.        root.left.right = new Node(5);

```

```

65.
66.     root.right.left = new Node(6);
67.     root.right.right = new Node(7);
68.
69.     zigzag(root);
70. }
71. }
72.

```

◆ 5. Check Identical Trees

👉 Dono trees same hain ya nahi

```

1. package Day_42;
2.
3. class IdenticalTree {
4.
5.     // Node class
6.     static class Node {
7.         int data;
8.         Node left, right;
9.
10.        Node(int data) {
11.            this.data = data;
12.            left = right = null;
13.        }
14.    }
15.
16.    // Function to check identical trees
17.    static boolean isIdentical(Node t1, Node t2) {
18.        if (t1 == null && t2 == null) return true;
19.
20.        if (t1 == null || t2 == null) return false;
21.
22.        return (t1.data == t2.data &&
23.            isIdentical(t1.left, t2.left) &&
24.            isIdentical(t1.right, t2.right));
25.    }
26.
27.    public static void main(String[] args) {
28.
29.        /*
30.            Tree 1          Tree 2
31.            1              1
32.           / \           / \
33.          2   3         2   3
34.        */
35.
36.        // Tree 1
37.        Node t1 = new Node(1);
38.        t1.left = new Node(2);
39.        t1.right = new Node(3);
40.
41.        // Tree 2
42.        Node t2 = new Node(1);
43.        t2.left = new Node(2);
44.        t2.right = new Node(3);
45.
46.        if (isIdentical(t1, t2))
47.            System.out.println("Trees are Identical");
48.        else
49.            System.out.println("Trees are NOT Identical");
50.    }
51. }
52.

```

🔥 Practice Input (Dry Run)

```
    1
   /\
  2 3
 /\ /
4 5 6
```

Expected Outputs:

Left View → 1 2 4

Right View → 1 3 6

ZigZag →

```
[1]
[3, 2]
[4, 5, 6]
```